Assignment 3: Basic Post-Quantum Cryptography

ComS/CprE 4590X/5590 2026 Spring

Total: 30 points
Due: **Friday, March 6, 11:59 PM**

1 Application Scenario

In this assignment, we consider a secure file sharing scenario. A sender (Alice) securely sends a text file to a receiver (Bob). The system must satisfy:

1. Confidentiality of file content.
2. Integrity protection of ciphertext.
3. No pre-shared secret key.
4. Public keys are known in advance.
5. Sender authenticity via digital signature.

You must implement the programs using:

- `liboqs-python`
- `cryptography`

2 Cryptographic Algorithms

Your implementation must use the following NIST standardized algorithms:

- **ML-KEM-768** (NIST standardized Kyber) for key encapsulation.
- **ML-DSA-65** (NIST standardized Dilithium) for digital signatures.
- **AES-128-GCM** for authenticated encryption.

3 Program 1: `KyberKeyGen.py` (5 pts)

Purpose

Generate an ML-KEM-768 public/private key pair.

Command Line Usage

```
python KyberKeyGen.py <public_key_file> <private_key_file>
```

Example

```
python KyberKeyGen.py BobKEMPub.data BobKEMPri.data
```

Required Tasks

1. Use ML-KEM-768.
2. Store keys as raw byte arrays.
3. Print public and private key sizes.

4 Program 2: `DilithiumKeyGen.py` (5 pts)

Purpose

Generate an ML-DSA-65 public/private key pair for digital signatures.

Command Line Usage

```
python DilithiumKeyGen.py <public_key_file> <private_key_file>
```

Example

```
python DilithiumKeyGen.py AliceDSAPub.data AliceDSAPri.data
```

Required Tasks

1. Use **ML-DSA-65** (NIST standardized Dilithium).
2. Generate a public/private key pair.
3. Store the public key in `<public_key_file>`.
4. Store the private key in `<private_key_file>`.
5. Keys must be stored as raw byte arrays.

Program Output

Print:

- Algorithm name (ML-DSA-65)
- Public key size (in bytes)
- Private key size (in bytes)

5 Program 3: `EncFile.py` (10 pts)

Purpose

Encrypt a plaintext file using:

- ML-KEM-768 (key establishment)
- AES-128-GCM (encryption)
- ML-DSA-65 (digital signature)

Command Line Usage

```
python EncFile.py <sender_dsa_private_key>
                  <receiver_kem_public_key>
                  <plaintext_file>
                  <ciphertext_file>
```

Example

```
python EncFile.py AliceDSAPri.data BobKEMPub.data plaintext.txt ciphertext
   .data
```

Required Steps

(a) Key Encapsulation

1. Read receiver's ML-KEM public key.
2. Perform encapsulation to obtain:
   - KEM ciphertext
   - Shared secret
3. Derive AES-128 key as:

```
AES_key = SHA256(shared_secret)[0:16]
```

(b) AES-GCM Encryption

1. Generate a 12-byte random nonce.
2. Encrypt plaintext with AES-128-GCM.
3. Obtain:
   - AES ciphertext
   - 16-byte authentication tag

(c) Digital Signature

Sign the concatenation:

```
KEM_ciphertext || nonce || AES_ciphertext || tag
```

using ML-DSA-65.

(d) Output File Format

The ciphertext file MUST contain:

```
[4 bytes KEM ciphertext length]
[KEM ciphertext]
[12 bytes nonce]
[AES ciphertext]
[16 bytes tag]
[ML-DSA signature]
[4 bytes signature length]   <-- stored LAST
```

Storing the signature length at the end ensures deterministic parsing.

Program Output

Print:

- Encryption complete
- KEM ciphertext length
- AES ciphertext length
- Signature length

6 Program 4: `DecFile.py` (10 pts)

Purpose

Verify authenticity and decrypt the file.

Command Line Usage

```
python DecFile.py <receiver_kem_private_key>
                  <sender_dsa_public_key>
                  <ciphertext_file>
```

Example

```
python DecFile.py BobKEMPri.data AliceDSAPub.data ciphertext.data
```

Required Steps

(a) Parse File

Extract:

- KEM ciphertext
- Nonce
- AES ciphertext
- Tag
- Signature

Signature length is obtained from the final 4 bytes.

(b) Signature Verification

Verify ML-DSA signature over:

```
KEM_ciphertext || nonce || AES_ciphertext || tag
```

If verification fails:

- Print "Signature verification FAILED."
- Terminate immediately.

(c) Key Decapsulation

1. Use ML-KEM private key to decapsulate.
2. Recover shared secret.
3. Derive AES key using SHA-256.

(d) AES-GCM Decryption

1. Decrypt using AES-128-GCM.
2. If authentication fails, terminate.
3. Print recovered plaintext.

Program Output

Print:

- Whether signature verification succeeded
- Decrypted plaintext

7 Submission Requirements

Submit exactly:

- `KyberKeyGen.py`
- `DilithiumKeyGen.py`
- `EncFile.py`
- `DecFile.py`

Programs must:

- Contain inline comments.
- Run in pyrite.
- Use ML-KEM-768, ML-DSA-65 and AES.

8 Grading Breakdown (30 pts)

Component	**Points**
ML-KEM-768 key generation (KyberKeyGen.py)	5
ML-DSA-65 key generation (DilithiumKeyGen.py)	5
Key encapsulation & AES-128-GCM encryption (EncFile.py)	5
ML-DSA signing logic & correct file format (EncFile.py)	5
Signature verification & key decapsulation (DecFile.py)	5
AES-GCM decryption & correctness checks (DecFile.py)	5
Total	**30**